

생활 맞춤형 식단 및 체중 관리 어플리케이션

< 개발 제안서 >

제안 사업명	청춘 일상 다이어트 총력전
제안자 (개발자)	오은찬 (정유운)
제안 일자	2026년 9월 9일

1. 제안 개요 및 개발 목표

• 1.1 제안 목적 및 개발 방향

- 건강 및 체중 관리를 위해 섭취한 음식과 체중 변화를 일일 별로 기록하고 검토할 필요가 있다. 하지만 이 둘을 따로 기록하는 경우 하루 섭취량과 체중 변화 간의 관계를 파악하기가 어렵다는 문제가 있다. 따라서 날짜별로 식단과 체중을 하나의 프로그램에서 관리하고 해당 내역을 이용하여 계산된 수치를 토대로 자신의 식습관과 체중 변화를 쉽게 확인할 수 있는 프로그램을 개발하고자 한다
- 사용자가 날짜별 식단기록과 해당 내역을 활용한 수치를 한 눈에 알아볼 수 있는 GUI 화면을 구축하고 사용자가 인터페이스 상에서 데이터를 동적으로 다룰 수 있도록 한다.

• 1.2 주요 개발 목표

- [] 사용자가 요구한 필수 기능 전부 구현
- [] 가독성이 높은 GUI 인터페이스 구현
- [] 예외 처리를 통한 프로그램 정상 동작
- [] JSON 파일 연동을 통한 데이터 영속성 및 일관성 확보

2. 개발 대상 및 환경 제안

- 대상 플랫폼 : [●] GUI / [] 웹 / [] CLI (콘솔) / [] 기타
- 사용 프로그래밍 언어 : Python (3.14)
- 데이터 저장 방식 : [●] JSON / [] Text / [] CSV / [] Database
- 기타 개발 제약조건 반영 계획 :
 - Python 내장 GUI 라이브러리 Tkinter 사용
 - 데이터베이스 사용 금지 (프로젝트 규정 준수)
 - 외부 라이브러리 사용 최소화 (프로젝트 규모에 맞지 않은 복잡한 기능 금지)

3. 상세 요구사항 구현 계획

• 3.1 데이터 요구사항 및 구조 설계

- 데이터 영속성 확보 방안
 - 프로그램 실행 중 데이터 입력 처리 시 동시에 저장까지 할 수 있도록 한다.
- 관리할 데이터 구조 설계

데이터 이름	데이터 설명	예시
날짜별 식단 기록	날짜별로 식단과 체중을 저장한 기록	{ 날짜 : { 체중, 식단 }, ... }
날짜	식단 및 체중 기록을 구분하는 날짜	2026-09-09
체중	해당 날짜에 측정한 체중	92.3 (kg)
식사 시간	음식을 섭취한 시간대 구분	아침, 점심, 저녁, 야식
음식 정보	음식에 대한 정보를 저장한 기록	{ ID : [이름, 칼로리], ... }
음식 ID	각 음식을 구분하기 위한 식별 번호	1
음식 이름	섭취한 음식의 이름	밥
섭취 칼로리	해당 음식의 섭취 칼로리	300 (kcal)
목표 칼로리	하루 섭취 목표 칼로리	1800 (kcal)
최근 체중 동향	7일전부터 오늘까지의 체중	[(날짜, 체중), ...]

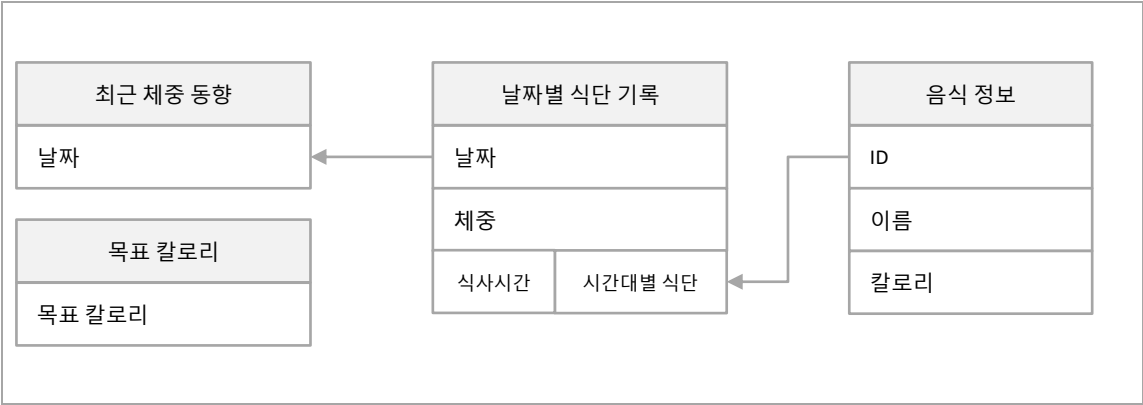
■ 프로그램 실행 중 동적 생성

- 실제 프로그램 사용 데이터 설계

데이터 이름	데이터 설명
날짜별 식단 기록	- 딕셔너리 구조 - 날짜, 체중, 음식, 식사시간을 저장
예시	
- Records.json { "2026-01-01" : { "weight" : 80, "foodlist" : { "breakfast" : [1, 2], "lunch" : [3, 4], "dinner" : [5, 6], "night" : [7, 8] } } }	

데이터 이름	데이터 설명
음식 정보	- 딕셔너리 구조 - 음식 ID, 음식 이름, 섭취 칼로리를 저장
예시	
Foods.json <pre>{ "1" : {"name" : "밥", "kcal" : 111 }, "2" : {"name" : "계란", "kcal" : 67 } }</pre>	

○ 데이터 간 관계 도식



• 3.2 기능 요구사항 구현 전략

요구사항 ID	기능명	구현 방안 및 로직
FR-01	식단 기록 및 관리	날짜를 선택하고 식사 시간대 별로 먹은 음식들을 추가한 후 해당 내역에 대한 CRUD가 가능해야 한다.
FR-02	체중 기록 및 변화 조회	현재 날짜부터 최근 7일 간의 체중 기록을 조회하여 순서대로 출력하고, 가장 오래된 체중 기록과 현재 체중을 비교하여 체중의 변화를 수치로 환산하여 출력한다.
FR-03	일일 섭취 칼로리 계산	현재 날짜를 기준으로 섭취한 음식의 칼로리를 모두 합하여 하루 총 섭취 칼로리를 출력한다.
FR-04	다이어트 총력전 리포트	일일 섭취 칼로리와 목표 칼로리를 비교하여 목표 달성 여부를 판단하고 이를 출력한다. 위 필수 기능에 대한 화면 GUI를 구현함으로써 하루 동안의 식단 및 체중 관리 결과를 종합적으로 제공한다.
FR-05	음식 검색 기능	식단 기록에서 음식을 추가할 때 검색하는 문자와 매칭되는 음식을 출력한다.

• 3.3 예외 처리 대응 방안

- [EH-01] 빈 값 입력 :
 - 데이터 입력 과정에서 값을 입력하지 않는 경우 해당 데이터에 대하여 기본값을 적용하거나 해당 데이터 처리를 취소하고 재입력 안내 메시지를 호출한다.
- [EH-02] 잘못된 값 입력 :
 - 데이터 입력 과정에서 데이터 허용 범위를 벗어난 값을 입력하거나 데이터 타입에 맞지 않는 경우 오류 메시지를 출력한다.
- [EH-03] 존재하지 않는 기록 조회 방지:
 - 입력한 날짜 또는 음식 ID에 해당하는 기록이 존재하는 경우만 인터페이스에 출력한다. 만약 조회가 불가능한 데이터를 조회를 시도할 경우 해당 작업을 중단하고 안내 메시지를 출력한다.
- [EH-04] 이벤트 오류 방지 :
 - 기능 처리에 대한 상호작용 버튼을 누르면 이벤트가 발생하는데 해당 이벤트가 여러 번 호출 될 경우 기능이 충돌하여 오류를 발생시킨다. 따라서 기능이 정상 수행할 수 있도록 버튼 이벤트를 구성한다.

- **3.4 비기능 요구사항 대응 방안**

- **GUI 화면 가독성 :**

- 식단 체중, 일일 칼로리, 리포트 외 기타 항목을 화면 영역별로 구분하여 사용자가 각 정보를 한 눈에 확인 할 수 있도록 구성한다.

- **안내 및 오류 메시지 :**

- 기능 처리 실패 시 사용자가 원인을 이해할 수 있도록 구체적인 안내 메시지를 출력한다.

- **응답 속도 :**

- 날짜 별 식단 기록 및 음식 데이터 처리, 칼로리 계산 외 연산 기능이 사용자가 데이터 입력 후 즉시 결과를 확인할 수 있게 동작하여야 한다.

4. 사용 시나리오 및 입출력 정의

- **4.1 사용 시나리오**

- (1) Main.py를 통한 프로그램 실행
 - (2) 실행 초기화 중 오늘 기준 기록 조회 및 초기 인터페이스 출력
 - 오늘 식단을 추가하지 않은 경우 안내 메시지 출력
 - (3) 식단 등록
 - (4) 날짜 선택 및 음식 추가
 - 예외가 발생하는 경우 해당 기능 처리 중단 및 안내 메시지 출력
 - (5) 해당 날짜 식단 등록 완료
 - (6) 오늘자를 기준으로 사용자에게 의한 기능 수행 및 리포트 인터페이스 출력
 - (7) 프로그램 종료

- **4.2 입출력 정의**

- (1) 주요 입력 및 출력
 - 날짜별 식단 기록 추가 ▶ 날짜, 음식, 해당 날짜 기준 몸무게 입력 ▶ 예외가 발생하는 경우 안내 메시지 출력 ▶ 문제가 없는 경우 데이터 처리
 - 음식 추가 ▶ 음식의 이름, 칼로리 입력 ▶ 예외가 발생하는 경우 안내 메시지 출력 ▶ 문제가 없는 경우 데이터 처리 및 식단 기록에 반영

(2) 예상 GUI 인터페이스 및 실행 순서도

☆ 청춘 일상 다이어트 총력전 ☆

날짜 : 2026-09-05

조회

음식명 : 밥

섭취 칼로리 : 300 kcal

식사 시간 : 점심

식단 등록

식단 삭제

식단 기록

아침 : 계란 / 150 kcal

점심 : 밥 / 300 kcal

저녁 : 없음

아식 : 없음

체중

체중 : 92.3 kg

체중 저장

일일 총 섭취 칼로리

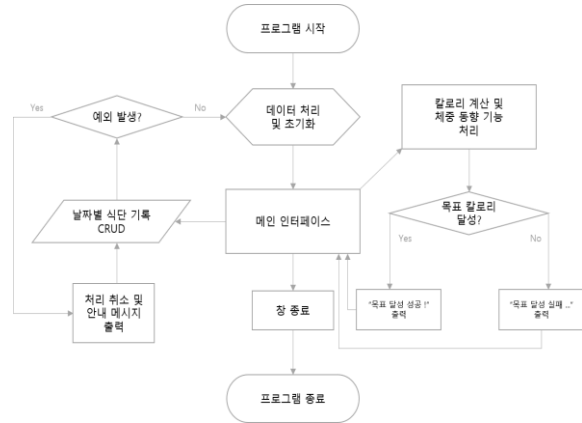
450 / 1800 kcal

최근 체중 변화

7일 전 93.1 kg → 현재 92.3 kg
0.8 kg 감소

결과

목표 달성 성공!



5. 프로젝트 관리

- 개발자는 '기획 및 요구사항 정의서' 및 '설계서'를 통하여 발주자와 협의 할 수 있도록 한다.
- 개발자는 구현 중 필요 시 발주자를 테스트에 참여시킬 수 있도록 한다.
- 개발자는 구현 중 문제가 발생하는 경우 발주자에게 시간 내 연락을 취할 수 있어야 한다.
- 발주자는 필요 시 개발자에게 요구 사항을 변경할 수 있도록 한다.
- 발주자와 개발자 사이에 개발 진행 상황을 주기적으로 점검할 수 있도록 한다.

6. 최종 산출물 제출 계획

GitHub 링크에서 아래와 같은 소스 파일들을 포함하도록 한다.

(1) 소스 코드 (Source Code) : 실행 가능한 최종 프로그램 코드

(2) 실제 데이터 파일 및 데이터 샘플 : 실제로 프로그램 내에 참조하는 데이터 파일 및 테스트용 파일

(3) 사용자 가이드 (README.md) : 프로그램 실행 방법 및 기능 설명서

→ 프로젝트 소개

→ 실행 방법

→ 주요 기능 설명

→ 파일 저장 구조 설명

→ 테스트 방법

7. 검수 및 자체 테스트 계획

- [] 필수 요구사항이 정상 동작 하는가?
- [] 예외 상황을 발생 시켜도 프로그램이 정상적으로 실행되는가?
- [] 프로그램 실행 간에 필수 데이터가 무결성을 유지하는가?
- [] 사용자 시점에서 소스 코드가 가독성이 있고 쉽게 이해할 수 있는가?
- [] 사용자가 원하는 GUI 인터페이스를 구성하였는가?
- [] 오류가 발생했을 때 사용자가 원인을 이해할 수 있는 안내 메시지가 제공되는가?
- [] 사용자가 불편하지 않게 프로그램 응답 속도를 빠르게 하였는가?

• 7.1 세부 테스트 케이스 [□ / 100%]

- Case 1 : 몸무게 입력란에 숫자와 관계 없는 문자열 삽입
- Case 2 : 오토 마우스를 사용하여 버튼 여러 번 누르기